

P0260R19

C++ Concurrent Queues

Published Proposal, 2025-06-18

This version:

<http://wg21.link/P0260R19>

Authors:

Lawrence Crowl

Chris Mysen

[Detlef Vollmann](#)

Gor Nishanov

Audience:

LWG

Project:

ISO/IEC 14882 Programming Languages — C++, ISO/IEC JTC1/SC22/WG21

Abstract

Concurrent queues are a fundamental structuring tool for concurrent programs. We propose concurrent queue concepts and a concrete implementation.

Table of Contents

1	Acknowledgments
2	Revision History
2.1	Review Topics
2.1.1	LWG
3	Introduction

4 Existing Practice

4.1 Concept of a Bounded Queue

4.2 Bounded and Unbounded Queues with C++ Interface

4.2.1 Literature

4.2.2 Boost

4.2.3 TBB

5 Examples and Implementation

5.1 Implementation

5.2 Examples

5.3 Find Files with String

5.4 Find Files with String (Async Coroutine Version)

5.5 Find Files with String (Async S/R Version)

5.6 Logging

6 Conceptual Interface

6.1 Status Enum

6.2 Basic Operations

6.3 Non-Waiting Operations

6.4 Asynchronous Operations

6.5 Closed Queues

6.6 `emplace`

6.7 Element Type Requirements

6.8 "Error" Handling

6.9 Single-Ended Interfaces

7 Concrete Queues

7.1 Movability

7.2 Memory Order Guarantees

8 Proposed Wording

8.1 Header `<version>` synopsis [version.syn]

8.2 Concurrent Queues [conqueues]

8.2.1 General [conqueues.general]

8.2.2 Header `<conqueue>` synopsis [conqueues.syn]

- 8.2.3 Exposition-only Concurrent Queue Concept [conqueue.concept]
- 8.2.3.1 Basic Concurrent Queue Concept [conqueue.concept.basic]
- 8.2.3.2 Concurrent Queue Concept [conqueue.concept.concurrent]
- 8.2.3.3 Asynchronous Queue Concept [conqueue.concept.async]
- 8.2.4 Class template `bounded_queue` [bounded.queue]
- 8.2.4.1 General [bounded.queue.general]
- 8.2.4.2 Constructor [bounded.queue.ctor]
- 8.2.4.3 Destructor [bounded.queue.dtor]
- 8.2.4.4 Allocator [bounded.queue.alloc]
- 8.2.4.5 Modifiers [bounded.queue.modifiers]

9 Old Revision History

10 Historic Contents

- 10.1 Abandoned Interfaces
 - 10.1.1 Re-opening a Queue
 - 10.1.2 Non-Blocking Operations
 - 10.1.3 Push Front Operations
 - 10.1.4 Queue Names
 - 10.1.5 Lock-Free Buffer Queue
 - 10.1.6 Storage Iterators
 - 10.1.7 Empty and Full Queues
 - 10.1.8 Queue Ordering
 - 10.1.9 Lock-Free Implementations
- 10.2 Abandoned Additional Conceptual Tools
 - 10.2.1 Fronts and Backs
 - 10.2.2 Streaming Iterators
 - 10.2.3 Binary Interfaces
 - 10.2.4 Managed Indirection
- 10.3 API Discussions
 - 10.3.1 `try_push(T&&, T&)`
 - 10.3.2 `try_pop` Queue Status Return
 - 10.3.2.1 Drain the queue without blocking

References

Informative References

§ 1. Acknowledgments

Thanks to David Goldblatt, Daniel Krügler, Dietmar Kühl, Jens Maurer and Gonzalo Brito for their help with the wording!

§ 2. Revision History

This paper revises P0260R1 - 2025-06-18 as follows:

- renamed `std::conqueue_errc` to `std::conqueue_status` as decided by LEWG in Sofia

P0260R18 revises P0260R17 - 2025-04-15 as follows:

- fixed wording based on feedback from Daniel Krügler
- added a paragraph in the design section that clearly states that the concepts don't require FIFO ordering
- added missing note for unfairness of unblocking pop operations
- aligned spurious failure wording with `mutex::try_lock`
- added note about acquire/release SC-for-DRF

P0260R17 revises P0260R16 - 2025-02-18 as follows:

- added wording to specify the unblocking of waiters
- added more explanation for `busy_async`

P0269R16 revises P0260R15 - 2025-02-13 as follows:

- removed exception support helpers
- removed discussion points and possible LEWG polls
- changed `try_pop` to not take an output parameter but return `expected`
- moved discussion for `expected` to historic contents
- introduced order `Q` for sequential consistency
- added wording to make clear that a value is removed from a queue if the constructor called by a pop operation throws an exception
- added list with wording still to be reviewed

P0260R15 revises P0260R14 - 2025-01-12 as follows:

- added more introductory description for the proposed wording
- updated examples and design section to reflect `set_error` for `async_push` and `async_pop`
- provided generally more rationale
- added wording for stop tokens
- added possible polls for LEWG
- fixed wording for `pop`
- removed `strongly happens before` for pop operations
- added `busy_async`
- added `get_await_completion_adapter_t` from P3570 with *error-as-optional* to return an `optional<T>` for coroutines from `async_pop`
- added *error-as-bool* to return a `bool` for coroutines from `async_push`

P0260R14 revises P0260R13 - 2024-12-10 as follows.

- updated wording based on feedback from Dietmar and Jens
- added example for non-blocking functions
- added discussion for `std::expected`
- added discussion for async sender behaviour
- wording now proposes for `async_push` and `async_pop` to call `set_error` on a closed queue

P0260R13 revises P0260R12 - 2024-11-21 as follows.

- added `success` to `conqueue_errc`
- updated error handling
- added rationale for concepts
- updated API (error handling, return types, `emplace`), including rationale
- updated examples

P0260R12 revises P0260R11 - 2024-10-12 as follows.

- Implemented feedback from LEWG and SG1 in St. Louis:
 - Added wording for allocator awareness
 - Added rationale for not providing single-ended interfaces
 - Added rationale for `bounded_queue` not being movable

- Added rationale for `bounded_queue` not providing `emplace`
- Implemented feedback from SG1 in Wroclaw:
 - Fixed wording for sequential consistency
 - Fixed wording for `async_*`
 - Fixed ordering specification of `bounded_queue`
- Provided usage examples

P0260R11 revises P0260R10 - 2024-06-26 as follows.

- Implement feedback from LEWG and SG1 in St. Louis:
- Make concept exposition only
- Require async operations to run on the scheduler of the receiver of the operation
- Call `set_error` for async operations if queue is closed
- Reintroduced accidentally removed data-race freeness from R9
- Require sequential consistent semantics for `bounded_queue`
- Remove `experimental` leftovers

P0260R10 revises P0260R9 - 2024-05-19 as follows.

- Implement feedback from SG1 in St. Louis:
- Split concept into three
- Require `try_*` to be lockfree (w/ error code `busy`)
- Drop `is_always_lock_free`
- Drop `capacity()`
- Removed discussion points for SG1
- Removed TS ship vehicle
- General cleanup in the design part

P0260R9 revises P0260R8 - 2024-03-08 as follows.

- Added more wording
- Fixed `capacity`
- Changed push/pop wording to use "strongly happens before"

- Added discussion points
- Removed paragraph about ctors/dtors returning on same thread
- `try_*` now never blocks (even for contention on internal synchronization)
- Wording for spurious failures of `try_*`
- The constructors for `bounded_queue` that pre-fill the queue have been removed
- Reference to P3282 added
- Moved discussion about `try_push(T &&, T &)` to historic contents.

P0260R8 revises P0260R7 - 2023-06-15 as follows.

- Restructured document
- Fix typos
- Implement LEWG feedback to derive `conqueue_errc` from `system_error`
- Implement LEWG feedback to add range constructor and go back to `InputIterator`
- `size_t` `capacity()` added
- Added TBB `concurrent_bounded_queue` as existing practice
- Moved discussion about `pop()` interface to separate paper
- reintegrated P1958 `buffer_queue`
- Renamed `buffer_queue` to `bounded_queue`
- Added proposed wording (incomplete)
- Updated TS questions

Older revision history was moved to [after the proposed wording](#).

§ 2.1. Review Topics

§ 2.1.1. LWG

Wording that still need to be reviewed by LWG:

- all

§ 3. Introduction

Queues provide a mechanism for communicating data between components of a system.

The existing `deque` in the standard library is an inherently sequential data structure. Its reference-returning element access operations cannot synchronize access to those elements with other queue operations. So, concurrent pushes and pops on queues require a different interface to the queue structure.

Moreover, concurrency adds a new dimension for performance and semantics. Different queue implementation must trade off uncontended operation cost, contended operation cost, and element order guarantees. Some of these trade-offs will necessarily result in semantics weaker than a serial queue.

Concurrent queues come in a several different flavours, e.g.

- bounded vs. unbounded
- blocking vs. overwriting
- single-ended vs. multi-ended
- strict FIFO ordering vs. priority based ordering

The syntactic concepts proposed here should be valid for all of these flavours, while the concrete semantics might differ.

§ 4. Existing Practice

§ 4.1. Concept of a Bounded Queue

The basic concept of a bounded queue with potentially blocking push and pop operations is very old and widely used. It's generally provided as an operating system level facility, like other concurrency primitives.

POSIX 2001 has `mq` message queues (with priorities and timeout).

FreeRTOS, Mbed, vxWorks provide bounded queues.

§ 4.2. Bounded and Unbounded Queues with C++ Interface

§ 4.2.1. Literature

The first concurrent queue I've seen was in [Hughes97]. It was full of bugs and as such shows what will go wrong if C++ doesn't provide a standard queue. It's unbounded.

Anthony Williams provided a queue in *C++ Concurrency in Action*. It's unbounded.

§ 4.2.2. Boost

Boost has a number of queues, as official library and as example uses of synchronization primitives.

Boost Message Queue only transfers bytes, not objects. It's bounded.

Boost Lock-Free Queue and *Boost Lock-Free SPSC Queue* have only non-blocking operations.

Boost Lock-Free Queue is bounded or unbounded, *Boost Lock-Free SPSC Queue* is bounded.

Boost Synchronized Queue is an implementation of an early version of this proposal.

§ 4.2.3. TBB

TBB has `concurrent_bounded_queue` (*TBB Bounded Queue*) and an unbounded version `concurrent_queue` *TBB Unbounded Queue*.

§ 5. Examples and Implementation

§ 5.1. Implementation

A partial implementation is available at github.com/GorNishanov/conqueue.

Another partial implementation is available at gitlab.com/cppzs/bounded-queue.

A free, open-source implementation of an earlier version of these interfaces is available at the Google Concurrency Library project at github.com/alasdairmackintosh/google-concurrency-library. The original `buffer_queue` is in `.../blob/master/include/buffer_queue.h`. The concrete `lock_free_buffer_queue` is in `.../blob/master/include/lock_free_buffer_queue.h`. The corresponding implementation of the conceptual tools is in `.../blob/master/include/queue_base.h`.

§ 5.2. Examples

Here we provide some examples how the API may be used.

The examples are available at <https://gitlab.com/cppzs/bounded-queue/-/tree/master/demo>.

§ 5.3. Find Files with String

This example was presented in *Object-Oriented Multithreading Using C++*. The program searches for files that contain a specific string. The idea is to have one thread to collect the file names from the filesystem and have several threads that read these files and searches them for the string.

Here we have some global definitions:

```
namespace fs = std::filesystem;
typedef std::bounded_queue<fs::path> FileList;
```

Here's the function that searches for the files and pushes them into a queue:

```
void searchFiles(FileList &files, fs::path dir)
{
    if (fs::exists(dir) && fs::is_directory(dir))
    {
        fs::directory_iterator end;
        for (fs::directory_iterator i(dir); i != end; ++i)
        {
            if (!fs::is_directory(*i))
            {
                files.push(*i);           // A
            }
        }
    }
    else
    {
        cerr << "no such directory" << endl;
    }

    files.close();                       // B
}
```

At *A* the files are pushed into the queue and after all files are pushed at *B* the queue is closed.

Here's the function that picks a file and searches it for the string:

```
void searchWord(FileList &files, string_view str)
{
    std::optional<fs::path> fname;

    while((fname = files.pop(ec))) // C
    {
        std::ifstream f(*fname);

        string line;
        while (getline(f, line))
        {
            if (line.find(str) != line.npos)
            {
                cout << "found in " << fname->string() << endl;
                break;
            }
        }
    }
}
```

Don't check for the queue being closed at *C*. Instead `pop`, and if the queue was closed *and no files are still in the queue* the returned optional will be empty.

And here's the `main` function to put everything together:

```
int main(int argc, char *argv[])
{
    FileList files(100); // D

    std::thread a(searchFiles, std::ref(files), argv[2]);
    std::thread b1(searchWord, std::ref(files), argv[1]);
    std::thread b2(searchWord, std::ref(files), argv[1]);

    a.join();
    b1.join();
    b2.join();
}
```

```
    return 0;
}
```

The queue is created (*D*) before the threads are started.

§ 5.4. Find Files with String (Async Coroutine Version)

NOTE: this async version is only presented to demonstrate the use of the interface. Without an async `getline` and an async `directory_iterator` this async version doesn't make much sense.

For an async version (using coroutines) only very few things change:

`searchFiles` is now a coroutine returning `task<void>` that simply calls

```
co_await files.async_push(*i);
```

instead of

```
files.push(*i);
```

`searchWord` has a few more changes:

```
exec::task<void> searchWord(FileList &files,
                           stdexec::run_loop &loop,
                           string_view str)
{
    std::optional<fs::path> fname;

    while ((fname = co_await (files.async_pop())) // B
    {
        // ... same as before ...
    }

    loop.finish();
}
```

At *B* we now `co_await async_pop`.

We now get a `run_loop` as parameter (*A*) on which we call `finish` (*C*) once we're done.

`main` now sets up a `run_loop` and `async_scope` instead of starting threads:

```
int  
main(int argc, char *argv[])  
{  
    stdexec::run_loop loop;  
    exec::async_scope scope;  
  
    FileList files(100);  
  
    scope.spawn(stdexec::on(loop.get_scheduler(), searchFiles(files, argv[1], &loop)));  
    scope.spawn(stdexec::on(loop.get_scheduler(), searchWord(files, loop, argv[2], &loop)));  
  
    loop.run();  
    stdexec::sync_wait(scope.on_empty());  
  
    return 0;  
}
```

§ 5.5. Find Files with String (Async S/R Version)

This version looks pretty different.

The function that creates the sender to search for files looks like this:

```
template <class Case0, class Case1, class Case2>
exec::variant_sender<Case0, Case1, Case2>
branch(unsigned condition, Case0 sndr0, Case1 sndr1, Case2 sndr2)
{
    if (condition == 0) return std::move(sndr0);
    if (condition == 1) return std::move(sndr1);
    return std::move(sndr2);
}

unsigned iterSelect(fs::directory_iterator& i)
{
    unsigned ret = 2;
    if (i == fs::directory_iterator{})
    {
        ret = 0;
    }
}
```

```

    }
    else
    {
        if (fs::is_directory(*i)) ret = 1;

        ++i; // A
    }

    return ret;
}

stdexec::sender auto
searchForFiles(FileList& files, fs::path dir)
{
    return stdexec::let_value( // B
        stdexec::just(fs::directory_iterator(dir)),
        [&files] (fs::directory_iterator& i) // C
        {
            stdexec::sender auto nextFile = stdexec::let_value( // D
                stdexec::just(),
                [&i, &files]
                {
                    fs::path file;
                    if (i != fs::directory_iterator{})
                    {
                        file = *i; // E
                    }

                    return branch( // F
                        iterSelect(i),

                        // Sndr0: close // G
                        stdexec::then(stdexec::just(),
                            [&files]
                            {
                                std::cout << "closing\n";
                                files.close();
                                return true;
                            }
                        ),

                        // Sndr1: skip directory // H
                        stdexec::just(false),

```

```

        // Sndr2: push file // I
        stdexec::then(files.async_push(file),
                      [] { return false; }));

    });

    return exec::repeat_effect_until(nextFile); // J
}

```

`let_value` at B ensures that the `i` at C is alive throughout its whole scope, i.e. for all iterations of `repeat_effect_until` at J. `let_value` at D ensures that `file` at I is alive for a single iteration.

`branch` at F is a helper that puts the correct sender into `variant_sender`. The correct sender is selected in `iterSelect` that also increments our iterator. We have three cases:

- The iterator is at the end, then we close the queue (G).
- The iterator points to a directory, then we skip the entry (H).
- Otherwise we push the entry into the queue (I).

The closing sender returns `true` to end the `repeat_effect_until`, the others return `false` to continue.

The function that creates the sender to search a single file for a string looks like this:

```

stdexec::sender auto searchWord(FileList& files, std::string_view str)
{
    stdexec::sender auto popAndProcess =
        files.async_pop() // A
    | stdexec::then(
        [str] (const fs::path& fname) noexcept -> bool // B
        {
            //std::cout << "searching " << fname.string() << std::endl;
            std::ifstream f(fname);
            std::string line;
            while (std::getline(f, line)) {
                if (line.find(str) != line.npos) {
                    std::cout << "found in " << fname.string() << '\n';
                    break;
                }
            }
            return false;
        })
}

```

```

        | stdexec::upon_error(                                     // C
          [] (auto err) noexcept -> bool
          {
              return true;
          });

    return exec::repeat_effect_until(popAndProcess);
}

```

The sender created by `async_pop` sends either a `fs::path` or an error if the queue is empty and closed.

The first case is covered by the `then` algorithm at **B** and processes the file. The second case is covered by the `upon_error` algorithm at **C** and returns `true` to end the loop.

§ 5.6. Logging

This is an example from an embedded system where no blocking is allowed.

Logging messages can be pushed into a queue from all kind of contexts, while a background task regularly pulls from the queue and sends messages to a serial interface.

This is the push function:

```

void enqueueLogMessage(std::string &&msg)
{
    if (logQ.try_push(std::move(msg)) != conqueue_status::success)
    {
        ++lostLogMessages;
    }
}

```

This is the function that pulls the messages:

```

void outputLogMessage()
{
    static std::string bufferedString;

    unsigned busyCnt = 0;

```



```

while (outputBuffer.add(bufferedString) > 0) // returns free space
{
    auto val = logQ.try_pop();
    if (val)
    {
        bufferedString = std::move(*val);
        continue;
    }

    conqueue_status ec(val.error());
    assert(ec != conqueue_status::closed);
    if (ec == conqueue_status::busy)
    {
        ++busyCnt;
        if (busyCnt > 2) break;
    }

    break;
}
}

```

§ 6. Conceptual Interface

We provide basic queue operations, and then extend those operations to cover other important use cases.

Due to performance issues, the conceptual interface is split into three separate interfaces. The `basic_concurrent_queue` concept provides the interface `push` and `pop` and closing the queue. The `concurrent_queue` concept is based on `basic_concurrent_queue` and provides the additional interface `try_push` and `try_pop`. The `async_concurrent_queue` concept is also based on `basic_concurrent_queue` and provides the additional interface `async_push` and `async_pop`.

The concrete queue `bounded_queue` models all these concepts. `bounded_queue` guarantees FIFO order. The concepts deliberately don't. This allows for priority based queues, but also for specific implementation strategies that don't guarantee FIFO order.

There's no single queue implementation that can handle all use cases efficiently. Also not all use cases require both `try_*` and `async_*` interfaces, and only providing one of them allows for more efficient implementations. Therefore it's expected that there will be many different implementations of

these concepts with different trade-offs. Some of them might be standardized, most will be not. But the existence of the concepts allows for adapters (like [single-ended interfaces](#)) to be used with different implementations. LEWG requested in St. Louis to make these concepts exposition-only, as this proposal doesn't include any adapters using these concepts.

§ 6.1. Status Enum

We introduce an `enum class` to define the possible states that operations on the queue may return.

```
enum class conqueue_status { success, empty, full, closed, busy, busy_async
```

§ 6.2. Basic Operations

The essential solution to the problem of concurrent queuing is to shift to value-based operations, rather than reference-based operations.

The basic operations are:

```
bool queue::push(const T& x);  
bool queue::push(T&& x);  
template <typename... Args> bool enqueue(Args &&... as);
```

Pushes `x` onto the queue via copy, move or argument based construction (and possibly blocks). It returns `true` on success, and `false` if the queue is closed.

```
std::optional<T> queue::pop();
```

Pops a value from the queue via move construction into the return value. If the queue is empty and closed, returns `std::nullopt`. If queue is empty and open, the operation blocks until an element is available.

§ 6.3. Non-Waiting Operations

Waiting on a full or empty queue can take a while, which has an opportunity cost. Avoiding that wait enables algorithms to do other work rather than wait for a push on a full queue, and to do other work

rather than wait for a pop on an empty queue. More importantly, there are contexts with weakly parallel forward progress that don't allow blocking synchronization, but where pushing into and popping from a queue is still desired. Also, blocking can really kill performance.

For all these reasons we provide `try_` versions of push and pop operations. These operations will never block. If they would have to wait for internal synchronization the queue status is `conqueue_status::busy`. If they would have to schedule a continuation of an async operation the queue status is `conqueue_status::busy_async`.

`busy_async` is an unfortunate case. The problem is that notification of an event to an async operation either means to schedule the continuation of the operation or to run it. The latter is really not an option for `try_push` as it runs arbitrary work which is potentially blocking. But scheduling generally means to enqueue the continuation into some work list, and this may not be possible without blocking. The current scheduler interface doesn't have any way to detect that the scheduling would block, so the way out is simply to return `busy_async`. [*Non-Blocking Support for `std::execution` \(P3669\)*](#) tries to fix the problem in the scheduler interface. Then `busy_async` may not be needed anymore.

```
conqueue_status queue::try_push(const T& x);
conqueue_status queue::try_push(T&& x);
template <typename... Args> conqueue_status try_emplace(Args &&... as);
```

NOTE: the return type has changed from earlier versions. Returning a `bool` and providing the reason for a failure in an extra output parameter feels wrong.

If no object could be placed into the queue, returns the respective status. Otherwise, pushes the value onto the queue via copy, move or argument based construction and returns `conqueue_status::success`.

```
expected<T, conqueue_status> queue::try_pop();
```

If no object could be obtained from the queue, returns an `expected` with an `unexpected` that contains the queue status. Otherwise, pops the element from the queue via move construction into the `expected`.

This is the interface agreed on in LEWG in Hagenberg.

§ 6.4. Asynchronous Operations

```
sender auto queue::async_push(T& x);
sender auto queue::async_push(T&& x);
template <typename... Args> sender auto async_emplace(Args &&... as);

sender auto queue::async_pop();
```

These operations return a sender that will push or pop the element. The operations support cancellation and if the receiver is currently waiting on a push or pop operation and no longer interested in performing the operation, it should be removed from any waiting queues, if any, and be completed with `std::execution::set_stopped`.

Sender based async interfaces have two main ways of usage: native S/R with receivers and coroutines where the receivers are provided by the coroutine library.

In Wroclaw only the coroutine usage was shown and LEWG voted there for an interface that favours coroutines. The sender returned by `async_pop()` would call `set_value(optional<T>)` which mirrors the synchronous `pop()`.

Afterwards it was pointed out that this interface is suboptimal for native receivers that can have multiple `set_value()` overloads. So R12 proposed to call `set_value(T)` if a value was retrieved and `set_value(void)` otherwise.

One change from R12 as presented in a telecon in December 2024 is that the close case takes the `set_error` path and not `set_value(void)`. One reason for this is that if the queue is closed, the pop fails to provide a value. In many cases it's not really an error, but it's still some kind of failure, and using the `set_value` channel for this feels wrong.

The native sender/receiver usage could look like this:

```
files.async_pop()
| stdexec::then(
|   [str] (const fs::path& fname) noexcept
|   {
|       //...
|       return false;
|   })
| stdexec::upon_error(
|   [] (auto err) noexcept -> bool
```

```
{
    return true;
});
```

But a native sender/receiver usage could also provide an application specific receiver that now can handle the value case and the closed case completely separately.

Another reason for using `set_error` is a practical one. If `async_pop` returns a value you often want to handle this value using more than one pipeline stage:

```
files.async_pop()
| stdexec::then(
|   [str] (T1 v1) noexcept
|   {
|       //...
|       return T2{...};
|   })
| some_worker.async_consume()
| stdexec::upon_error(
|   [] (auto err) noexcept -> bool
|   {
|       return true;
|   });
```

If the closed case would call the `set_value` channel, it would be harder to write such chains, and it's much easier to handle all failure cases with a single `upon_error` at the end.

By symmetry, `async_push` (and `async_emplace`) also call `set_error(conqueue_status)` on a closed queue. For success they simply call `set_value(void)`.

[*P3570R0: optional variants in sender/receiver*](#) provides a mechanism to return different values for coroutines than for direct receivers. This revision proposes to use this mechanism.

For the coroutine use it provides an exposition-only adapter *error-as-optional* that returns an `optional<T>` that contains a value if a value was retrieved and an empty `optional<T>` if the queue was closed.

For `async_push` an exposition-only adapter *error-as-bool* is provided that returns for coroutines `true` if the push succeeded and `false` if the queue was closed.

§ 6.5. Closed Queues

Threads using a queue for communication need some mechanism to signal when the queue is no longer needed. The usual approach is add an additional out-of-band signal. However, this approach suffers from the flaw that threads waiting on either full or empty queues need to be woken up when the queue is no longer needed. To do that, you need access to the condition variables used for full/empty blocking, which considerably increases the complexity and fragility of the interface. It also leads to performance implications with additional mutexes or atomics. Rather than require an out-of-band signal, we chose to directly support such a signal in the queue itself, which considerably simplifies coding.

To achieve this signal, a thread may `close` a queue. Once closed, no new elements may be pushed onto the queue. Push operations on a closed queue will return `false` or `conqueue_status::closed`. Elements already on the queue may be popped. When a queue is empty and closed, pop operations will either (`pop`) return an empty `optional` or (`try_pop`) return an `expected` with an `unexpected` result with value `conqueue_status::closed`.

The additional operations are as follows:

```
void queue::close() noexcept;
```

Close the queue.

```
bool queue::is_closed() const noexcept;
```

Return true iff the queue is closed.

§ 6.6. `emplace`

As queues have a `push` operation, it might be plausible to add an `emplace` operation as well. However, this might be misleading. `emplace` typically exists for performance reason for types that don't provide cheap move construction. So for containers where objects are meant to stay it makes sense to construct the objects in place. But queues aren't containers. Queues are mechanisms to push objects into them to be popped out again as soon as possible. So if the move of an object is expensive, it might be efficient to construct it inside a queue, but the pop would still be expensive. In such cases it might be more efficient to push a `unique_ptr` or similar through a queue instead of the actual values.

Providing an `emplace` operation might lead users to use a queue with object values and believe that's efficient.

However, `try_emplace` is definitely useful as no object is created at all if it's not inserted into the queue.

And if `try_emplace` exists, for symmetry `emplace` and `async_emplace` should be provided as well, so this revision proposes them.

§ 6.7. Element Type Requirements

The above operations require element types with copy/move constructors, and destructor. These operations may be trivial. The copy/move constructors operators may throw, but must leave the objects in a valid state for subsequent operations.

§ 6.8. "Error" Handling

It's extremely hard to define what results of a queue function actually constitutes an error. Instead of trying so, this proposal takes the approach that nothing is really an error.

Instead different function results due to different queue states are delivered by special return types.

`pop()` returns `optional<T>` (as decided in Wroclaw): empty `optional` for the `closed` state, and an optional containing a `T` otherwise.

`push()` returns `bool`: `false` for `closed` and `true` for successful enqueueing.

`try_pop` returns `expected<T, conqueue_status>`: a `T` on success and the queue state otherwise. This was decided in LEWG in Hagenberg.

`try_push` returns as `conqueue_status` the queue state directly.

`async_pop()` returns a sender that calls `set_value(T)` if a value could be obtained from the queue and `set_error(conqueue_status)` if the queue was closed.

`async_push()` returns a sender that calls `set_value(void)` if the value could be deposited and `set_error(conqueue_status)` if the queue was closed.

Concurrent queues cannot completely hide the effect of exceptions thrown by the element type, in part because changes cannot be transparently undone when other threads are observing the queue.

Queues operations may rethrow exceptions from storage allocation or mutexes.

If the [element type operations required](#) do not throw exceptions, then only the exceptions above are rethrown. In practice, for a [bounded_queue](#) storage allocation only happens on construction and mutex [lock](#) and [unlock](#) generally only throw on deadlock detection.

When an element copy/move may throw, some queue operations have additional behavior.

- A push operation shall rethrow and the state of the queue is unaffected.
- A pop operation shall rethrow and the element is popped from the queue. The value popped is effectively lost. (Doing otherwise would likely clog the queue with a bad element.)

§ 6.9. Single-Ended Interfaces

In many applications one part only uses one end of a queue, while a different part uses the other end. I.e. producer and consumer are logically separated. So it makes a lot of sense to provide interface types that only provide either the push or the pop interface.

But providing such separate interfaces requires additional overhead to ensure there arise no lifetime issues. So requiring such interfaces is not part of the proposed concept.

And then it's assumed that a lot of different implementations of the concept will exist, with different tradeoffs. A separate type for the single-ended interfaces that works for all these implementation is more efficient in terms of implementation. Such a type is not part of this proposal but should be a separate proposal. And as separate type it can easily be added later.

§ 7. Concrete Queues

In addition to the concepts, the standard needs at least one concrete queue. This will not provide the most efficient implementation but serves all synchronization use cases. For this reason it's bounded to allow for slowing down producers that are too fast.

This paper proposes a fixed-size [bounded_queue](#). It meets all three concepts for queues. The constructor takes a parameter specifying the maximum number of elements in the queue and an optional allocator.

[bounded_queue](#) is only allowed to allocate in its constructor.

§ 7.1. Movability

The `bounded_queue` implementation will probably contain synchronization mechanisms like `std::mutex` and `std::condition_variable`. These are not movable, so `bounded_queue` isn't movable either.

§ 7.2. Memory Order Guarantees

`bounded_queue` provides sequentially consistent semantics.

The reasoning for this is that for a queue (as high-level data structure) provided by the IS usage safety should be more important than efficiency. Background on this is found in *Memory Model Issues for Concurrent Data Structures (P0387R1)* and *Concurrency Safety in C++ Data Structures (P0495)*.

The example from P0387R1 illustrates the problem clearly:

`q` and `log` are concurrent queues:

Thread 1	Thread 2
<code>q.push(1);</code>	<code>log.push("pushed 2");</code>
<code>log.push("pushed 1");</code>	<code>q.push(2);</code>

With only acquire/release this could result in `log` having "pushed 1" and "pushed 2", while `q` has 2 and 1.

`bounded_queue` will probably need a mutex based locking implementation anyway, which provides sequential consistency for free.

§ 8. Proposed Wording

Suggested location of concurrent queues in the standard is [thread].

§ 8.1. Header `<version>` synopsis [version.syn]

```
#define __cpp_lib_conqueue <editor supplied value> // also in <conqueue>
```

§ 8.2. Concurrent Queues [conqueues]

§ 8.2.1. General [conqueues.general]

Concurrent queues provide a mechanism to transfer objects from one point in a program to another without producing data races. Push operations insert objects into a queue from one execution context and pop operations retrieve them typically from another execution context. Push and pop operations can be blocking, non-blocking or asynchronous. If a queue gets closed, any subsequent push operations will fail with a respective queue status. Any pop operations after a close will succeed while there are still objects in the queue and fail afterwards. Any pending operations will be resumed on a close with a failure indication.

§ 8.2.2. Header `<conqueue>` synopsis [conqueues.syn]

```
namespace std {
    enum class conqueue_status {
        success = unspecified,
        empty = unspecified,
        full = unspecified,
        closed = unspecified,
        busy = unspecified,
        busy_async = unspecified
    };

    template <class Q>
        concept basic-concurrent-queue; // exposition only
    template <class Q>
        concept concurrent-queue; // exposition only
    template <class Q>
        concept async-concurrent-queue; // exposition only

    struct error-as-optional-t { see below }; // exposition only
    struct error-as-bool-t { see below }; // exposition only

    inline constexpr error-as-optional-t error-as-optional; // exposition only
    inline constexpr error-as-bool-t error-as-bool; // exposition only

    struct async-pop-env { // exposition only
        auto query(get_await_completion_adapter_t) const -> error-as-optional-t;
        return {};
    }
}
```

```

}
struct async-push-env { // exposition only
    auto query(get_await_completion_adapter_t) const -> error-as-bool-t {
        return {};
    }
}

template <class T, class Allocator = allocator<T>>
class bounded_queue;

namespace pmr {
    template<class T>
    using bounded_queue = std::bounded_queue<T, polymorphic_allocator<T>>;
}
}

```

§ 8.2.3. Exposition-only Concurrent Queue Concept [*conqueue.concept*]

§ 8.2.3.1. Basic Concurrent Queue Concept [*conqueue.concept.basic*]

1. The exposition-only concept *basic-concurrent-queue* defines the requirements for a basic queue type.

```

namespace std {
    template <class Q>
    concept basic-concurrent-queue = // exposition only
        move_constructible<typename Q::value_type> &&
        requires (Q q, const Q cq, typename Q::value_type &&t) {
            { cq.is_closed() } noexcept -> same_as<bool>;
            { q.close() } noexcept -> same_as<void>;
            { q.push(std::forward<typename Q::value_type>(t)) } -> same_as<
            { []<class... Args>(Q &q, Args... args) { bool b{q.emplace(std:
            { q.pop() } -> same_as<optional<typename Q::value_type>>;
        };
}

```

2. In the following description, *Q* denotes a type modeling the *basic-concurrent-queue* concept, *q* denotes an object of type *Q*, and *t* denotes an object convertible to *Q::value_type*.
3. The expression *q.is_closed()* has the following semantics:

1. *Returns:* `true` if the queue is closed, `false` otherwise.
4. The expression `q.close()` has the following semantics:
 1. *Effects:* Closes the queue.
5. `q.push(std::forward<T>(t))` and `q.emplace(std::forward<Args>(args)...)` are *push operations* and `q.pop()` is a *pop operation*.
6. A push operation *deposits* an object into a queue. Hereby it is made available to be *extracted* from the queue. A pop operation extracts and returns an object from a queue. *push-deposited* is the event when the object is made available. *pop-claimed* is the event when a pop operation decides to extract the object.
7. If an object `t` is deposited by a push operation and there is at least one pop operation blocked on `q`, one of those pop operations will be unblocked. [*Note:* This unblocking is not required to be fair.]
8. If an object `t` is extracted by a pop operation and there is at least one push operation blocked on `q`, one of those push operations will be unblocked. [*Note:* This unblocking is not required to be fair.]
9. If an object `t` is deposited by a push operation, the call of the constructor of this object inside the queue's storage strongly happens before the return of the pop operation that extracts `t`. A pop operation on a queue shall only extract objects that were deposited into the same queue and each object shall only be extracted once.

[*Note:* This doesn't specify when the *push-deposited* actually happens during the push operation, and likewise the *pop-claimed* during the pop operation. So it can happen that a pop operation already claimed a specific object but is blocked because the constructor of the object hasn't finished yet. -- end note]
10. Calls to operations (except constructor and destructor) on the same queue from different threads of execution do not introduce data races.
11. [*Note:* This concept doesn't specify whether a push may block for space available in the queue (bounded queue) or whether a push may allocate new space (unbounded queue). -- end note]
12. The expression `q.emplace(args...)` has the following semantics:
 1. *Effects:* If the queue is not closed, a value is deposited into `q`. In this case the value in the queue storage is direct-non-list-initialized with `std::forward<Args>(args)...`
 2. *Returns:*
 - `true` if `t` was deposited into `q`; `false` otherwise.

3. *Throws*: Any exception thrown by the selected constructor of `T`. An `emplace` may throw additional exceptions. If an exception is thrown, no value is deposited into `q`.
13. The expression `q.push(t)` is equivalent to `return q.emplace(std::forward<T>(t))`.
14. The expression `q.pop()` has the following semantics:
 1. *Effects*: Blocks the current thread until there is an object available in the queue or until the queue is closed. If there is an object available in the queue, extracts the object and returns an `optional<T>` with a move-constructed value. Otherwise, if the queue is closed returns a default constructed `optional<T>`.
 2. *Returns*: as specified above.
 3. *Throws*: Any exception thrown by the selected constructor of `T`. A `pop` may throw additional exceptions. Even if an exception is thrown, the value is extracted from `q`.

§ 8.2.3.2. Concurrent Queue Concept [`conqueue.concept.concurrent`]

1. The exposition-only concept *concurrent-queue* defines the requirements for a concurrent queue type.

```
namespace std {
    template <class Q>
        concept concurrent_queue = // exposition only
            basic_concurrent_queue<Q> &&
            requires (Q q, typename Q::value_type &t, conqueue_status ec) {
                { q.try_push(std::forward<typename Q::value_type>(t)) } noexcept
                { []<class... Args>(Q &q, Args... args) { conqueue_status e{q.t
                { q.try_pop() } noexcept(see below) -> same_as<expected<typenam
                };
            };
}
```

2. `try_push`, `try_emplace` and `try_pop` are `noexcept` if the selected constructor of `T` is `noexcept`.
3. In the following description, `Q` denotes a type modeling the *concurrent-queue* concept, `q` denotes an object of type `Q`, `t` denotes an object convertible to `Q::value_type` and `ec` denotes an object of type `conqueue_status`.
4. `try_push` and `try_emplace` are push operations if they deposit an object into the queue and `try_pop` is a pop operation if it extracts an object from a queue.

5. `try_push` or `try_emplace` may return `conqueue_status::full` even if the queue has space for the element. Similarly `try_pop` may return an `unexpected` with values `conqueue_status::empty` even if the queue has an element to be extracted. [Note: This spurious failure is normally uncommon but allows for more implementation options. -- end note] An implementation should ensure that these functions do not consistently spuriously fail in the absence of contending such operations.

6. The expression `q.try_emplace(t)` has the following semantics:

1. *Effects*: If the queue is not closed, and space is available in the queue, a value is deposited into `q`. In this case the value in the queue storage is direct-non-list-initialized with

`std::forward<Args>(args)...` without blocking.

2. *Returns*:

- `conqueue_status::success` if `t` was deposited into `q`,
- `conqueue_status::closed` if the queue is closed,
- `conqueue_status::full` if the queue doesn't have space,
- `conqueue_status::busy` if the operation would block for internal synchronization,
- `conqueue_status::busy_async` if the queue also models the *async-concurrent-queue* concept, the operation would have to schedule the continuation of an `async_pop`, and the implementation can not detect if this scheduling would block for internal synchronization of the scheduler.

[Note: An implementation may return consistently `conqueue_status::busy_async` if there are only `async_pop` operations waiting.]

3. *Throws*: Any exception thrown by the selected constructor of `T`. A `try_emplace` may throw additional exceptions. If an exception is thrown, no value is deposited into `q`.

7. The expression `q.try_push(t)` is equivalent to `return q.try_emplace(std::forward<T>(t))`.

8. The expression `q.try_pop()` has the following semantics:

1. *Effects*: If there is an object available in the queue it will be extracted from the queue and returned without blocking.

2. *Returns*: An object of type `expected<T, conqueue_status>`. The return value will contain a move-constructed value of type `T` if a value could be extracted. Otherwise an `unexpected` will be returned with the value:

- `conqueue_status::closed` if the queue is closed,

- `conqueue_status::empty` if the queue doesn't have an object available,
- `conqueue_status::busy` if the operation would block for internal synchronization,
- `conqueue_status::busy_async` if the queue also models the *async-concurrent-queue* concept, the operation would have to schedule the continuation of an `async_push`, and the implementation can not detect if this scheduling would block for internal synchronization of the scheduler.

[Note: An implementation may return consistently `conqueue_status::busy_async` if there are only `async_push` operations waiting.]

3. *Throws*: Any exception thrown by the selected constructor of `T`. A `try_pop` may throw additional exceptions. Even if an exception is thrown, the value is extracted from `q`.

§ 8.2.3.3. Asynchronous Queue Concept [*conqueue.concept.async*]

1. The exposition-only concept *async-concurrent-queue* defines the requirements for an asynchronous queue type.

```
namespace std {
    template <class Q>
        concept async-concurrent-queue = // exposition only
            async-concurrent-queue<Q> &&
            requires (Q q, typename Q::value_type &&t) {
                { q.async_push(std::forward<typename Q::value_type>(t)) } noexcept
                { q.async_emplace(std::forward<typename Q::value_type>(t)) } noexcept
                { []<class... Args>(Q &q, Args... args) { q.async_emplace(std::forward<Args>(args)) } } noexcept
                { q.async_pop() } noexcept;
            };
}
```

2. The exposition only name *error-as-optional* denotes a pipeable sender adaptor object. For a sub-expression `sndr`, let `Sndr` be `decltype((sndr))`. The expression `error-as-optional(sndr)` is expression-equivalent to:

```
transform_sender(
    get-domain-early(sndr),
    make_sender(error-as-optional, {}, sndr))
```

except that `sndr` is only evaluated once.

3. Let `sndr` and `env` be subexpressions such that `Sndr` is `decltype((sndr))` and `Env` is `decltype((env))`. The expression `error-as-optional.transform_sender(sndr, env)` is equivalent to:

```
auto&& [_, _, child] = sndr;
return let_error(
    then(std::forward_like<Sndr>(child),
        [] (T &&) noexcept (is_nothrow_move_constructible_v<T>) {
            return optional<T>(in_place, std::move(t));
        }),
    [] (auto e) noexcept {
        if constexpr (is_same_v<decltype(e), conqueue_status>) {
            return just(optional<T>());
        } else {
            return just_error(e);
        }
    });
```

4. The exposition only name `error-as-bool` denotes a pipeable sender adaptor object. For a sub-expression `sndr`, let `Sndr` be `decltype((sndr))`. The expression `error-as-bool(sndr)` is expression-equivalent to:

```
transform_sender(
    get-domain-early(sndr),
    make_sender(error-as-bool, {}, sndr))
```

except that `sndr` is only evaluated once.

5. Let `sndr` and `env` be subexpressions such that `Sndr` is `decltype((sndr))` and `Env` is `decltype((env))`. The expression `error-as-bool.transform_sender(sndr, env)` is equivalent to:

```
auto&& [_, _, child] = sndr;
return let_error(
    then(std::forward_like<Sndr>(child),
        [] () noexcept { return just(true); }),
    [] (auto e) noexcept {
        if constexpr (is_same_v<decltype(e), conqueue_status>) {
            return just(false);
        } else {
            return just_error(e);
        }
    });
```



```

    }
  });
  []() noexcept { return false; });

```

6. In the following description, `Q` denotes a type modeling the *async-concurrent-queue* concept, `q` denotes an object of type `Q` and `t` denotes an object convertible to `Q::value_type`.
7. `async_push` and `async_emplace` are push operations and `async_pop` is a pop operations.
8. The expression `q.async_emplace(args...)` has the following semantics:
 1. Let `w` be a sender object returned by the expression, `op` be an operation state obtained from connecting `w` to a receiver `r` and `st` be a stop token returned from `get_stop_token(get_env(w))`.
 2. *Returns:* A sender object `w` that behaves as follows:
 1. After `op.start()` is called, a completion function of `r` is called when there s space in the queue or when the queue is closed.
 2. If `st.stop_requested()` returns `true` `set_stopped(r)` will be called.
 3. If there is space in the queue `t` will be deposited into the queue and `set_value(r)` will be called. The value in the queue storage is direct-non-list-initialized with `std::forward<Args>(args)...`.
 4. If the selected constructor throws an exception, `set_error(r, current_exception())` will be called. No value will be deposited into `q`.
 5. If the queue is closed `set_error(r, conqueue_status::closed)` will be called.
 6. `set_value()`, `set_stopped()` or `set_error()` will be called on the scheduler of `r`.
 7. `get_env(w)` returns an object of tye *async-push-env*.
9. The expression `q.async_push(t)` is equivalent to `return q.async_emplace(std::forward<T>(t))`.
10. The expression `q.async_pop()` has the following semantics:
 1. Let `w` be a sender object returned by the expression, `op` be an operation state obtained from connecting `w` to a receiver `r` and `st` be a stop token returned from `get_stop_token(get_env(w))`.
 2. *Returns:* A sender object `w` that behaves as follows:

1. After `op.start()` is called, a completion function of `r` is called when there is an object available in the queue or when the queue is closed.
2. If `st.stop_requested()` returns `true` `set_stopped(r)` will be called.
3. If there is an object `t` available in the queue it will be extracted and `set_value(r, std::move(t))` will be called.
4. If the selected constructor for `set_value` throws an exception, `set_error(r, current_exception())` will be called. The value will be extracted from the queue nevertheless.
5. If the queue is closed `set_error(r, conqueue_status::closed)` will be called.
6. `set_value()`, `set_stopped()` or `set_error()` will be called on the scheduler of `r`.
7. `get_env(w)` returns an object of type *async-pop-env*.

§ 8.2.4. Class template `bounded_queue` [`bounded.queue`]

§ 8.2.4.1. General [*bounded.queue.general*]

1. A `bounded_queue` models *concurrent-queue* and *async-concurrent-queue* and can hold a fixed number of objects which is given at construction time.

```
2. template <class T,
    class Allocator = allocator<T>>
class bounded_queue
{
    bounded_queue(bounded_queue&&) = delete;

public:
    using value_type = T;
    using allocator_type = Allocator;

    // construct/destroy
    explicit bounded_queue(size_t max_elems, const Allocator& alloc = {})

    ~bounded_queue();

    // observers
    bool is_closed() const noexcept;
```

```

allocator_type get_allocator() const;

// modifiers
void close() noexcept;

bool push(const T& x);
bool push(T&& x);
template bool emplace(Args &&... as);

conqueue_status try_push(const T& x);
conqueue_status try_push(T&& x);
template conqueue_status try_emplace(Args &&... as);

sender auto async_push(const T&);
sender auto async_push(T&&);
template sender auto async_emplace(Args &&... as);

optional<T> pop();
expected<T, conqueue_status> try_pop();
sender auto async_pop();

private:
    allocator_type alloc;          // exposition only
};

```

3. `T` shall be a type that meets the *Cpp17Destructible* requirements (Table [tab:cpp17.destructible]).
4. Template argument `Allocator` shall satisfy the *Cpp17Allocator* requirements ([allocator.requirements]). An instance of `Allocator` is maintained by the `bounded_queue` object during the lifetime of the object. The allocator instance is set at `bounded_queue` object creation time.
5. The values inside the storage of the queue deposited by push operations are constructed using `allocator_traits<Allocator>::construct(alloc, p, std::forward<T>(x))`, where `p` is the address of the element being constructed.

§ 8.2.4.2. Constructor [bounded.queue.ctor]

```
explicit bounded_queue(size_t max_elems, const Allocator& alloc = Allocator())
```

6. *Effects*: Constructs an object with no elements, but with storage for `max_elems`. The storage for the elements will be allocated using `alloc`.

7. *Remarks:* The operations of `bounded_queue` will not allocate any memory outside the constructor.

§ 8.2.4.3. Destructor [*bounded.queue.dtor*]

```
~bounded_queue();
```

8. *Effects:* Destroys all objects in the queue and deallocates the storage for the elements using `alloc`.

§ 8.2.4.4. Allocator [*bounded.queue.alloc*]

```
allocator_type get_allocator() const;
```

1. *Returns:* A copy of the `Allocator` that was passed to the object's constructor.

§ 8.2.4.5. Modifiers [*bounded.queue.modifiers*]

1. The push and pop operations of `bounded_queue` provide sequentially consistent semantics.

```
void push(const T& x);  
void push(T&& x);  
template <class... Args> bool emplace(Args &&... as);
```

2. *Effects:* Blocks the current thread until there is space in the queue or until the queue is closed.

3. Let *Push1* and *Push2* be push operations and *Pop1* and *Pop2* be pop operations, where *Pop1* returns the value of the parameter given to *Push1* and *Pop2* returns the value of the parameter given to *Push2*. There is a total order *Q* shared by all `bounded_queue` objects consistent with the single total order *S* of `memory_order::seq_cst` operations [atomics.order]p4 of *push-deposited* and *pop-claimed* of *Push1*, *Push2*, *Pop1* and *Pop2*, and moreover if *push-deposited* of *Push1* is before *push-deposited* of *Push2* in that order, then *pop-claimed* of *Pop1* is before *pop-claimed* of *Pop2* in that order. [Note: This paragraph's intent is enabling implementations based on acquire and release operations under the SC-for-DRF theorem. --end note]

4. It is unspecified whether the constructors and destructors of the objects in the internal storage of the queue run under a lock of the queue implementation.
5. [Note: This guarantees FIFO behaviour, but for two concurrent pushes the constructors cannot determine the order in which the values are enqueued, and the constructors can run concurrently as well. -- end note]
6. [Note: This does not guarantee that constructors or destructors may ever run concurrently. An implementation may decide that two pushes (or two pops) never run concurrently. -- end note]
7. [Note: A constructor or destructor can deadlock if it tries a push or pop on the same queue. -- end note]

```
T pop();
optional<T> pop();
expected<T, conqueue_status> try_pop();
sender auto async_pop();
```

8. *Effects*: The object in the queue is destroyed using `allocator_traits<Allocator>::destroy`.
9. *Remarks*: The constructor of the returned object and the destructor of the internal object run on the same thread of execution.

```
bool is_closed() const noexcept;
void close() noexcept;
conqueue_status try_push(const T& x);
conqueue_status try_push(T&& x);
template <class... Args> conqueue_status try_emplace(Args &&... as);
sender auto async_push(const T&);
sender auto async_push(T&&);
template <class... Args> sender auto async_emplace(Args &&... as);
```

10. These functions behave as described in the respective concepts.

§ 9. Old Revision History

P0260R6 revises P0260R5 - 2023-01-15 as follows.

- Fixing typos.
- Added a scope for the target TS.

- Added questions to be answered by a TS.
- Added asynchronous interface

P0260R5 revises P0260R4 - 2020-01-12 as follows.

- Added more introductory material.
- Added response to feedback by LEWGI at Prague meeting 2020.
- Added section on existing practice.
- Replaced `value_pop` with `pop`.
- Replaced `is_lock_free` with `is_always_lockfree`.
- Removed `is_empty` and `is_full`.
- Added move-into parameter to `try_push(Element&&)`
- Added note that exception thrown by the queue operations themselves are derived from `std::exception`.
- Added a note that the wording is partly invalid.
- Moved more contents into the "Abandoned" part to avoid confusion.

P0260R4 revised P0260R3 - 2019-01-20 as follows.

- Remove the binding of `queue_op_status::success` to a value of zero.
- Correct stale use of the `Queue` template parameter in `shared_queue_front` to `Value`.
- Change the return type of `share_queue_ends` from a `pair` to a custom struct.
- Move the concrete queue proposal to a separate paper, [\[P1958\]](#).

P0260R3 revised P0260R2 - 2017-10-15 as follows.

- Convert `queue_wrapper` to a `function`-like interface. This conversion removes the `queue_base` class. Thanks to Zach Lane for the approach.
- Removed the requirement that element types have a default constructor. This removal implies that statically sized buffers cannot use an array implementation and must grow a vector implementation to the maximum size.
- Added a discussion of checking for output iterator end in the wording.
- Fill in synopsis section.
- Remove stale discussion of `queue_owner`.
- Move all abandoned interface discussion to a new section.

- Update paper header to current practice.

P0260R2 revised P0260R1 - 2017-02-05 as follows.

- Emphasize that non-blocking operations were removed from the proposed changes.
- Correct syntax typos for `noexcept` and template alias.
- Remove `static` from `is_lock_free` for `generic_queue_back` and `generic_queue_front`.

P0260R1 revised P0260R0 - 2016-02-14 as follows.

- Remove pure virtuals from `queue_wrapper`.
- Correct `queue::pop` to `value_pop`.
- Remove nonblocking operations.
- Remove non-locking buffer queue concrete class.
- Tighten up push/pop wording on closed queues.
- Tighten up push/pop wording on synchronization.
- Add note about possible non-FIFO behavior.
- Define `buffer_queue` to be FIFO.
- Make wording consistent across attributes.
- Add a restriction on element special methods using the queue.
- Make `is_lock_free()` for only non-waiting functions.
- Make `is_lock_free()` static for non-indirect classes.
- Make `is_lock_free()` `noexcept`.
- Make `has_queue()` `noexcept`.
- Make destructors `noexcept`.
- Replace "throws nothing" with `noexcept`.
- Make the remarks about the usefulness of `is_empty()` and `is_full` into notes.
- Make the non-static member functions `is_...` and `has_...` functions `const`.

P0260R0 revised N3533 - 2013-03-12 as follows.

- Update links to source code.
- Add wording.

- Leave the name facility out of the wording.
- Leave the push-front facility out of the wording.
- Leave the reopen facility out of the wording.
- Leave the storage iterator facility out of the wording.

N3532 revised N3434 = 12-0124 - 2012-09-23 as follows.

- Add more exposition.
- Provide separate non-blocking operations.
- Add a section on the lock-free queues.
- Argue against push-back operations.
- Add a cautionary note on the usefulness of `is_closed()`.
- Expand the cautionary note on the usefulness of `is_empty()`. Add `is_full()`.
- Add a subsection on element type requirements.
- Add a subsection on exception handling.
- Clarify ordering constraints on the interface.
- Add a subsection on a lock-free concrete queue.
- Add a section on content iterators, distinct from the existing streaming iterators section.
- Swap front and back names, as requested.
- General expository cleanup.
- Add an "Revision History" section.

N3434 revised N3353 = 12-0043 - 2012-01-14 as follows.

- Change the inheritance-based interface to a pure conceptual interface.
- Put `try_...` operations into a separate subsection.
- Add a subsection on non-blocking operations.
- Add a subsection on push-back operations.
- Add a subsection on queue ordering.
- Merge the "Binary Interface" and "Managed Indirection" sections into a new "Conceptual Tools" section. Expand on the topics and their rationale.
- Add a subsection to "Conceptual Tools" that provides for type erasure.

- Remove the "Synopsis" section.
- Add an "Implementation" section.

§ 10. Historic Contents

The Contents in this section is for historic reference only.

§ 10.1. Abandoned Interfaces

§ 10.1.1. Re-opening a Queue

There are use cases for opening a queue that is closed. While we are not aware of an implementation in which the ability to reopen a queue would be a hardship, we also imagine that such an implementation could exist. Open should generally only be called if the queue is closed and empty, providing a clean synchronization point, though it is possible to call open on a non-empty queue. An open operation following a close operation is guaranteed to be visible after the close operation and the queue is guaranteed to be open upon completion of the open call. (But of course, another close call could occur immediately thereafter.)

```
void queue::open();
```

Open the queue.

Note that when `is_closed()` returns false, there is no assurance that any subsequent operation finds the queue closed because some other thread may close it concurrently.

If an open operation is not available, there is an assurance that once closed, a queue stays closed. So, unless the programmer takes care to ensure that all other threads will not close the queue, only a return value of true has any meaning.

Given these concerns with reopening queues, we do not propose wording to reopen a queue.

§ 10.1.2. Non-Blocking Operations

For cases when blocking for mutual exclusion is undesirable, one can consider non-blocking operations. The interface is the same as the try operations but is allowed to also return

`queue_op_status::busy` in case the operation is unable to complete without blocking.

```
queue_op_status queue::nonblocking_push(const Element&);\ queue_op_status  
queue::nonblocking_push(Element&&);
```

If the operation would block, return `queue_op_status::busy`. Otherwise, if the queue is full, return `queue_op_status::full`. Otherwise, push the `Element` onto the queue. Return `queue_op_status::success`.

```
queue_op_status queue::nonblocking_pop(Element&);
```

If the operation would block, return `queue_op_status::busy`. Otherwise, if the queue is empty, return `queue_op_status::empty`. Otherwise, pop the `Element` from the queue. The element will be moved out of the queue in preference to being copied. Return `queue_op_status::success`.

These operations will neither wait nor block. However, they may do nothing.

The non-blocking operations highlight a terminology problem. In terms of synchronization effects, `nonwaiting_push` on queues is equivalent to `try_lock` on mutexes. And so one could conclude that the existing `try_push` should be renamed `nonwaiting_push` and `nonblocking_push` should be renamed `try_push`. However, at least Thread Building Blocks uses the existing terminology. Perhaps better is to not use `try_push` and instead use `nonwaiting_push` and `nonblocking_push`.

****In November 2016, the Concurrency Study Group chose to defer non-blocking operations. Hence, the proposed wording does not include these functions. In addition, as these functions were the only ones that returned `busy`, that enumeration is also not included.****

§ 10.1.3. Push Front Operations

Occasionally, one may wish to return a popped item to the queue. We can provide for this with `push_front` operations.

```
void queue::push_front(const Element&);\ void queue::push_front(Element&&);
```

Push the `Element` onto the back of the queue, i.e. in at the end of the queue that is normally popped. Return `queue_op_status::success`.

```
queue_op_status queue::try_push_front(const Element&);\ queue_op_status  
queue::try_push_front(Element&&);
```

If the queue was full, return `queue_op_status::full`. Otherwise, push the `Element` onto the front of the queue, i.e. in at the end of the queue that is normally popped. Return `queue_op_status::success`.

```
queue_op_status queue::nonblocking_push_front(const Element&);\nqueue_op_status queue::nonblocking_push_front(Element&&);
```

If the operation would block, return `queue_op_status::busy`. Otherwise, if the queue is full, return `queue_op_status::full`. Otherwise, push the `Element` onto the front queue. i.e. in at the end of the queue that is normally popped. Return `queue_op_status::success`.

This feature was requested at the Spring 2012 meeting. However, we do not think the feature works.

- The name `push_front` is inconsistent with existing `"push back"` nomenclature.
- The effects of `push_front` are only distinguishable from a regular push when there is a strong ordering of elements. Highly concurrent queues will likely have no strong ordering.
- The `push_front` call may fail due to full queues, closed queues, etc. In which case the operation will suffer contention, and may succeed only after interposing push and pop operations. The consequence is that the original push order is not preserved in the final pop order. So, `push_front` cannot be directly used as an `'undo'`.
- The operation implies an ability to reverse internal changes at the front of the queue. This ability implies a loss efficiency in some implementations.

In short, we do not think that in a concurrent environment `push_front` provides sufficient semantic value to justify its cost. Consequently, the proposed wording does not provide this feature.

§ 10.1.4. Queue Names

It is sometimes desirable for queues to be able to identify themselves. This feature is particularly helpful for run-time diagnostics, particularly when `'ends'` become dynamically passed around between threads. See [Managed Indirection](#).

```
const char* queue::name();
```

Return the name string provided as a parameter to queue construction.

There is some debate on this facility, but we see no way to effectively replicate the facility. However, in recognition of that debate, the wording does not provide the name facility.

§ 10.1.5. Lock-Free Buffer Queue

We provide a concrete concurrent queue in the form of a fixed-size `lock_free_buffer_queue`. It meets the `NonWaitingConcurrentQueue` concept. The queue is still under development, so details may change.

In November 2016, the Concurrency Study Group chose to defer lock-free queues. Hence, the proposed wording does not include a concrete lock-free queue.

§ 10.1.6. Storage Iterators

In addition to iterators that stream data into and out of a queue, we could provide an iterator over the storage contents of a queue. Such an iterator, even when implementable, would mostly likely be valid only when the queue is otherwise quiescent. We believe such an iterator would be most useful for debugging, which may well require knowledge of the concrete class. Therefore, we do not propose wording for this feature.

§ 10.1.7. Empty and Full Queues

It is sometimes desirable to know if a queue is empty.

```
bool queue::is_empty() const noexcept;
```

Return true iff the queue is empty.

This operation is useful only during intervals when the queue is known to not be subject to pushes and pops from other threads. Its primary use case is assertions on the state of the queue at the end of its lifetime, or when the system is in quiescent state (where there are no outstanding pushes).

We can imagine occasional use for knowing when a queue is full, for instance in system performance polling. The motivation is significantly weaker though.

```
bool queue::is_full() const noexcept;
```

Return true iff the queue is full.

Not all queues will have a full state, and these would always return false.

§ 10.1.8. Queue Ordering

The conceptual queue interface makes minimal guarantees.

- The queue is not empty if there is an element that has been pushed but not popped.
- A push operation *synchronizes with* the pop operation that obtains that element.
- A close operation *synchronizes with* an operation that observes that the queue is closed.
- There is a sequentially consistent order of operations.

In particular, the conceptual interface does not guarantee that the sequentially consistent order of element pushes matches the sequentially consistent order of pops. Concrete queues could specify more specific ordering guarantees.

§ 10.1.9. Lock-Free Implementations

Lock-free queues will have some trouble waiting for the queue to be non-empty or non-full. Therefore, we propose two closely-related concepts. A full concurrent queue concept as described above, and a non-waiting concurrent queue concept that has all the operations except `push`, `wait_push`, `value_pop` and `wait_pop`. That is, it has only non-waiting operations (presumably emulated with busy wait) and non-blocking operations, but no waiting operations. We propose naming these `WaitingConcurrentQueue` and `NonWaitingConcurrentQueue`, respectively.

NOTE: Adopting this conceptual split requires splitting some of the facilities defined later.

For generic code it's sometimes important to know if a concurrent queue has a lock free implementation.

```
constexpr static bool queue::is_always_lock_free() noexcept;
```

Return true iff the has a lock-free implementation of the non-waiting operations.

§ 10.2. Abandoned Additional Conceptual Tools

There are a number of tools that support use of the conceptual interface. These tools are not part of the queue interface, but provide restricted views or adapters on top of the queue useful in implementing concurrent algorithms.

§ 10.2.1. Fronts and Backs

Restricting an interface to one side of a queue is a valuable code structuring tool. This restriction is accomplished with the classes `generic_queue_front` and `generic_queue_back` parameterized on the concrete queue implementation. These act as pointers with access to only the front or the back of a queue. The front of the queue is where elements are popped. The back of the queue is where elements are pushed.

```
void send( int number, generic_queue_back<buffer_queue<int>> arv );
```

These fronts and backs are also able to provide `begin` and `end` operations that unambiguously stream data into or out of a queue.

§ 10.2.2. Streaming Iterators

In order to enable the use of existing algorithms streaming through concurrent queues, they need to support iterators. Output iterators will push to a queue and input iterators will pop from a queue. Stronger forms of iterators are in general not possible with concurrent queues.

Iterators implicitly require waiting for the advance, so iterators are only supportable with the `WaitingConcurrentQueue` concept.

```
void iterate(
    generic_queue_back<buffer_queue<int>>::iterator bitr,
    generic_queue_back<buffer_queue<int>>::iterator bend,
    generic_queue_front<buffer_queue<int>>::iterator fitr,
    generic_queue_front<buffer_queue<int>>::iterator fend,
    int (*compute)( int ) )
{
    while ( fitr != fend && bitr != bend )
        *bitr++ = compute(*fitr++);
}
```

Note that contrary to existing iterator algorithms, we check both iterators for reaching their end, as either may be closed at any time.

Note that with suitable renaming, the existing standard front insert and back insert iterators could work as is. However, there is nothing like a pop iterator adapter.

§ 10.2.3. Binary Interfaces

The standard library is template based, but it is often desirable to have a binary interface that shields client from the concrete implementations. For example, `std::function` is a binary interface to callable object (of a given signature). We achieve this capability in queues with type erasure.

We provide a `queue_base` class template parameterized by the value type. Its operations are virtual. This class provides the essential independence from the queue representation.

We also provide `queue_front` and `queue_back` class templates parameterized by the value types. These are essentially `generic_queue_front<queue_base<Value>>` and `generic_queue_back<queue_base<Value>>`, respectively.

To obtain a pointer to `queue_base` from an non-virtual concurrent queue, construct an instance the `queue_wrapper` class template, which is parameterized on the queue and derived from `queue_base`. Upcasting a pointer to the `queue_wrapper` instance to a `queue_base` instance thus erases the concrete queue type.

```
extern void seq_fill( int count, queue_back<int> b );

buffer_queue<int> body( 10 /*elements*/, /*named*/ "body" );
queue_wrapper<buffer_queue<int>> wrap( body );
seq_fill( 10, wrap.back() );
```

§ 10.2.4. Managed Indirection

Long running servers may have the need to reconfigure the relationship between queues and threads. The ability to pass 'ends' of queues between threads with automatic memory management eases programming.

To this end, we provide `shared_queue_front` and `shared_queue_back` template classes. These act as reference-counted versions of the `queue_front` and `queue_back` template classes.

The `share_queue_ends(Args ... args)` template function will provide a pair of `shared_queue_front` and `shared_queue_back` to a dynamically allocated `queue_object` instance containing an instance of the specified implementation queue. When the last of these fronts and

backs are deleted, the queue itself will be deleted. Also, when the last of the fronts or the last of the backs is deleted, the queue will be closed.

```
auto x = share_queue_ends<buffer_queue<int>>>( 10, "shared" );
shared_queue_back<int> b(x.back);
shared_queue_front<int> f(x.front);
f.push(3);
assert(3 == b.value_pop());
```

§ 10.3. API Discussions

§ 10.3.1. `try_push(T&&, T&)`

REVISITED in Varna

The following version was introduced in response to LEWG-I concerns about losing the element if an rvalue cannot be stored in the queue.

```
queue_op_status queue::try_push(T&&, T&);
```

However, SG1 reaffirmed the APIs above with the following rationale:

It seems that it is possible not to lose the element in both versions:

```
T x = get_something();
if (q.try_push(std::move(x))) ...
```

With two parameter version:

```
T x;
if (q.try_push(get_something(), x)) ...
```

Ergonomically they are roughly identical. API is slightly simpler with one argument version, therefore, we reverted to original one argument version.

§ 10.3.2. `try_pop` Queue Status Return

This is the interface agreed on in St. Louis, based on discussion in [\[P2921R0\]](#):

POLL: LEWG would like to add a `std::expected` interface for concurrent queues.

SF	F	N	A	SA
0	2	5	3	2

However, [P2921R0] mainly compares `std::expected` vs. exceptions. Now concurrent queue operations don't throw own exceptions anymore (they might forward exceptions from constructors).

For `try_pop` [P2921R0] has the following example:

§ 10.3.2.1. Drain the queue without blocking

```
conqueue_errc ec;
while (auto val = q.try_pop(ec))
    println("got {}", *val);
if (ec == conqueue_errc::closed)
    return;
// do something else.
```

`std::expected` based has unfortunate duplication since we want to know why the pop failed and we have to move `val` out of the loop condition.

```
auto val = q.try_pop();
while (val) {
    println("got {}", *val);
    val = q.try_pop();
}
if (val.error() == conqueue_errc::closed)
    return;
// do something else
```

So for `while` loops with the `try_pop` inside the loop condition the separate output parameter has advantages.

The logging example above shows a case where `expected` might be considered to be superior. So it might make sense to revisit the decision from St. Louis.

§ References

§ Informative References

[BoostLFQ]

Boost Lock-Free Queue. URL:

https://www.boost.org/doc/libs/1_85_0/doc/html/boost/lockfree/queue.html

[BoostLFSPSCQ]

Boost Lock-Free SPSC Queue. URL:

https://www.boost.org/doc/libs/1_85_0/doc/html/boost/lockfree/spsc_queue.html

[BoostMQ]

Boost Message Queue. URL:

https://www.boost.org/doc/libs/1_85_0/doc/html/boost/interprocess/message_queue_t.html

[BoostSQ]

Boost Synchronized Queue. URL: https://www.boost.org/doc/libs/1_85_0/doc/html/thread/sd-s.html

[Hughes97]

Cameron Hughes; Tracey Hughes. *Object-Oriented Multithreading Using C++*.

[P0387R1]

Memory Model Issues for Concurrent Data Structures (P0387R1). URL:

<https://wg21.link/P0387R1>

[P0495]

Concurrency Safety in C++ Data Structures (P0495). URL: <https://wg21.link/P0495>

[P1958]

C++ Concurrent Buffer Queue (P1958). URL: <https://wg21.link/P1958>

[P2921R0]

Gor Nishanov, Detlef Vollmann. *Exploring std::expected based API alternatives for buffer_queue*. 5 July 2023. URL: <https://wg21.link/p2921r0>

[P3570]

P3570R0: optional variants in sender/receiver. URL: <https://wg21.link/P3570>

[P3669]

Non-Blocking Support for `std::execution` (P3669). URL: <https://wg21.link/P3668>

[TBBQB]

TBB Bounded Queue. URL: https://spec.oneapi.io/versions/latest/elements/oneTBB/source/containers/concurrent_bounded_queue_cls.html

[TBBQUB]

TBB Unbounded Queue. URL:

https://spec.oneapi.io/versions/latest/elements/oneTBB/source/containers/concurrent_queue_-

[cls.html](#)

[Williams17]

Anthony Williams. *C++ Concurrency in Action*.